

UC-PACT: UNIVERSAL COMPOSABILITY FOR PREVENTING ADVERSARIAL COMPOSITION TECHNIQUES

Modeling the Needham-Schroeder Public-Key Protocol in EasyUC

Robert Graham
14 Aug 2025

This material is based upon work supported by the Defense Advanced Research Projects Agency (DARPA) and Naval Information Warfare Center Pacific (NIWC Pacific) under N66001-22-C-4020.

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the DARPA and NIWC Pacific.



UC-PACT: UNIVERSAL COMPOSABILITY FOR PREVENTING ADVERSARIAL COMPOSITION TECHNIQUES

Needham-Schroeder through the
Decades: A Personal Reflection

Robert Graham

14 Aug 2025

OVERVIEW

- The Protocol
 - Notation for public-key cryptographic protocols
 - The Needham-Schroeder public-key protocol
 - Analyzing Needham-Schroeder
 - Attacks on Needham-Schroeder
- The Models
 - Untrusted network communication
 - EasyCrypt support theories
 - Needham-Schroeder
- Discussion
 - Analysis of the model
 - Alternative ideal functionalities
 - Revised EasyUC model of Needham-Schroeder
- Modeling the MITM attack
- References

Recurring themes

1. **What security goals are we trying to achieve?**
2. **How do we know we've achieved them?**

THE PROTOCOL

NOTATION FOR PUBLIC-KEY CRYPTOGRAPHIC PROTOCOLS

- Principals: **A**, **B**, ..., and the adversary, **Adv**
- Nonces
 - A randomly generated integer used “once” (e.g., per session)
 - **Na** denotes a nonce generated by principal A
- Key pairs
 - **Ka** denotes the public key for principal A; **Ka⁻¹** denotes the corresponding private key
- Encryption/decryption
 - Let **M** be any plaintext message, such as A, Na
 - **{M} Ka** is the encryption of M using A’s public key
 - Anyone can do the encryption
 - Only A can decrypt it and recover M: $\{ \{M\} Ka \} Ka^{-1} = M$
 - **{M} Ka⁻¹** is the signature containing M using A’s private key
 - Only A can sign M
 - Anyone can verify the signature and recover M: $\{ \{M\} Ka^{-1} \} Ka = M$

THE NEEDHAM-SCHROEDER PUBLIC-KEY PROTOCOL

- Needham & Schroeder, “Using Encryption for Authentication in Large Networks of Computers,” 1978
- Target functions
 - Authenticated interactive communication between two principals
 - Where *authenticated* means each principal has verified the identity of the other
 - Signed communication, in which the origin and integrity of a communication can be authenticated to a third party
- The adversary can alter or copy parts of messages, replay messages or emit false material, but cannot decrypt messages if it hasn’t seen the corresponding key, guess a key, etc.
- Two principals, A and B, plus a certificate authority, S, containing public credentials
 - 1a. A → S: A, B
 - 1b. S → A: {K_b, B} K_s⁻¹
 - 2a. A → B: {Na, A} K_b
 - 3a. B → S: B, A
 - 3b. S → B: {K_a, A} K_s⁻¹
 - 2b. B → A: {Na, Nb} K_a
 - 2c. A → B: {Nb} K_b

} A looks up B’s public key

} B looks up A’s public key

THE NEEDHAM-SCHROEDER PUBLIC-KEY PROTOCOL, CONT.

- Now that A and B have authenticated each other, how do they carry on a conversation?
 - Double encryption
 - A $\rightarrow$ B: $\{\{M\} K_a^{-1}\} K_b$ (or $\{A, \{M\} K_a^{-1}\} K_b$)
 - B $\rightarrow$ A: $\{\{M\} K_b^{-1}\} K_a$ (or $\{B, \{M\} K_b^{-1}\} K_a$)
 - Why bother with the protocol above, then?
 - Use the nonces (not clear from this paper or BAN89)
 - A $\rightarrow$ B: $\{N_b, M\} K_b$
 - B $\rightarrow$ A: $\{N_a, M\} K_a$

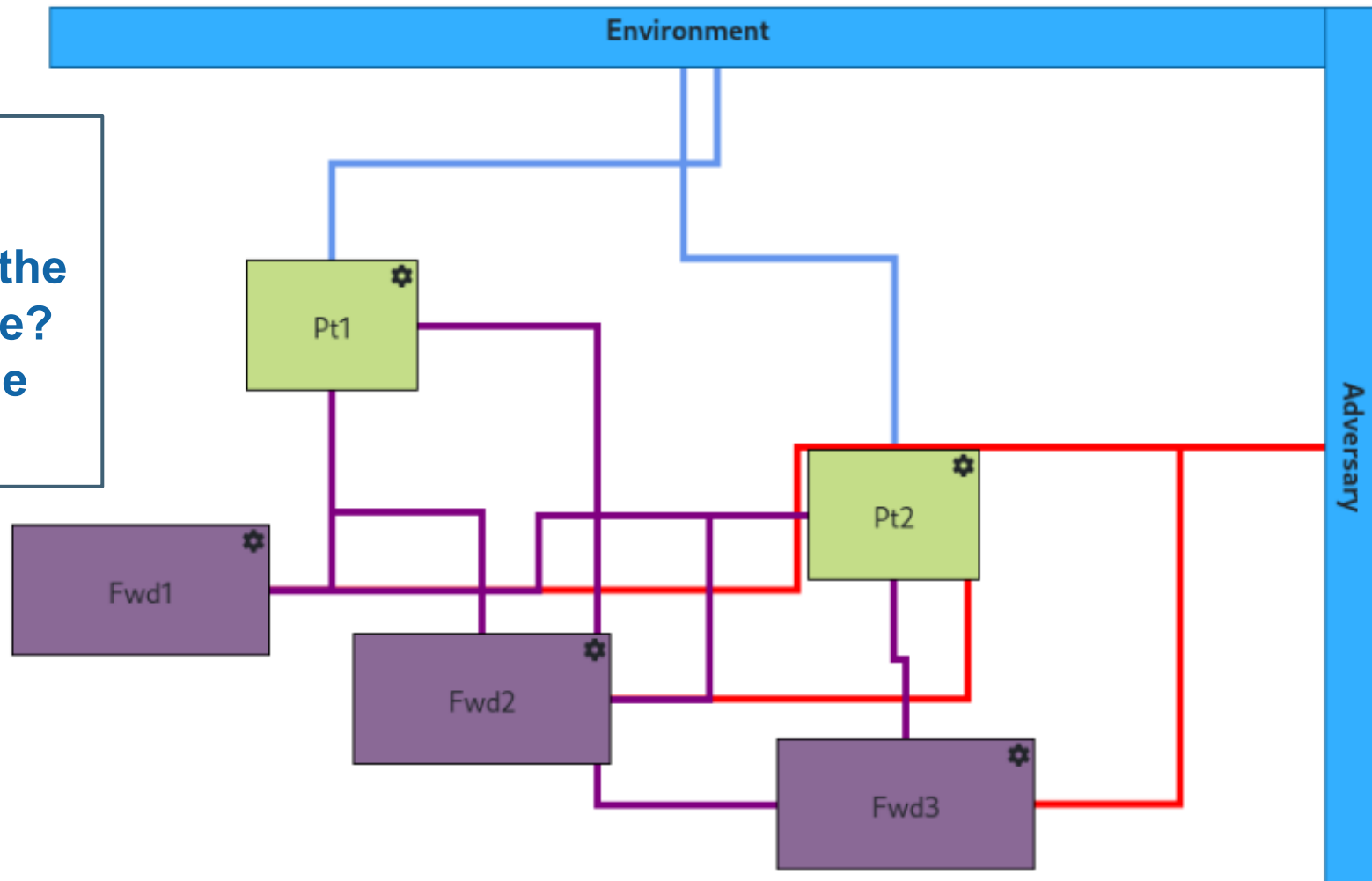
THE REAL WORLD FOR NEEDHAM-SCHROEDER

NSReal

This is easy!

...

**Except, what is the
direct interface?
And what will the
ideal be?**



ANALYZING NEEDHAM-SCHROEDER

- Burrows, Abadi and Needham, “A Logic of Authentication,” 1989
- What security properties does the Needham-Schroeder protocol guarantee?
 - A authenticates S (Message 1b)
 - K_b is bound to B (Message 1b)
 - A authenticates B (Message 2b)
 - B authenticates S (Message 3b)
 - K_a is bound to A (Message 3b)
 - B authenticates A (Message 2c)*
 - N_a and N_b are secrets between A and B (and trusted associates of A and B)
 - K_a is current (Message 1b)*
 - K_b is current (Message 2b)*
 - N_a is fresh (Message 2a)
 - N_b is fresh (Message 2b)

Which of these are essential and which are incidental to this particular protocol?

* Oops, not really

- The protocol further assumes that S and Ks are bound and known to A and B *a priori*

ATTACKS ON NEEDHAM-SCHROEDER

- The adversary can fool A (B) into accepting an old public key for B (A)

1a. A → S: A, B (Adv eavesdrops and saves)

1b. S → A: {B, K_b} K_s⁻¹ (Adv eavesdrops and saves)

... much later

1a'. A → S: A, B (Adv intercepts and replies)

1b'. Adv → A: {B, K_b} K_s⁻¹

- The adversary can't read the message, but can replay it

- The adversary can fool B into believing it is A (Lowe95)

2a. A → Adv: {N_a, A} K_{adv}

2a'. Adv → B: {N_a, A} K_b

2b'. B → Adv: {N_a, N_b} K_a

2b. Adv → A: {N_a, N_b} K_a

2c. A → Adv: {N_b} K_{adv}

2c'. Adv → A: {N_b} K_b

Result: A (B) uses a public key for B (A) that has expired or been revoked; B (A) may no longer have the corresponding secret key
Fix: Add a timestamp to Message 1b

Result (a “weird” machine):
A and Adv have a session
Adv and B have a session
but B thinks its session is with A
Both sessions use N_a and N_b
Fix: Change Message 2b to {N_a, N_b, B} K_a
(a.k.a. Needham-Schroeder-Lowe)

MODELING NEEDHAM-SCHROEDER WITH CSP

- Gavin Lowe, “Breaking and Fixing the Needham-Schroeder Public-Key Protocol using FDR,” 1996
- Defines Initiator, Responder and Intruder (not shown) processes:

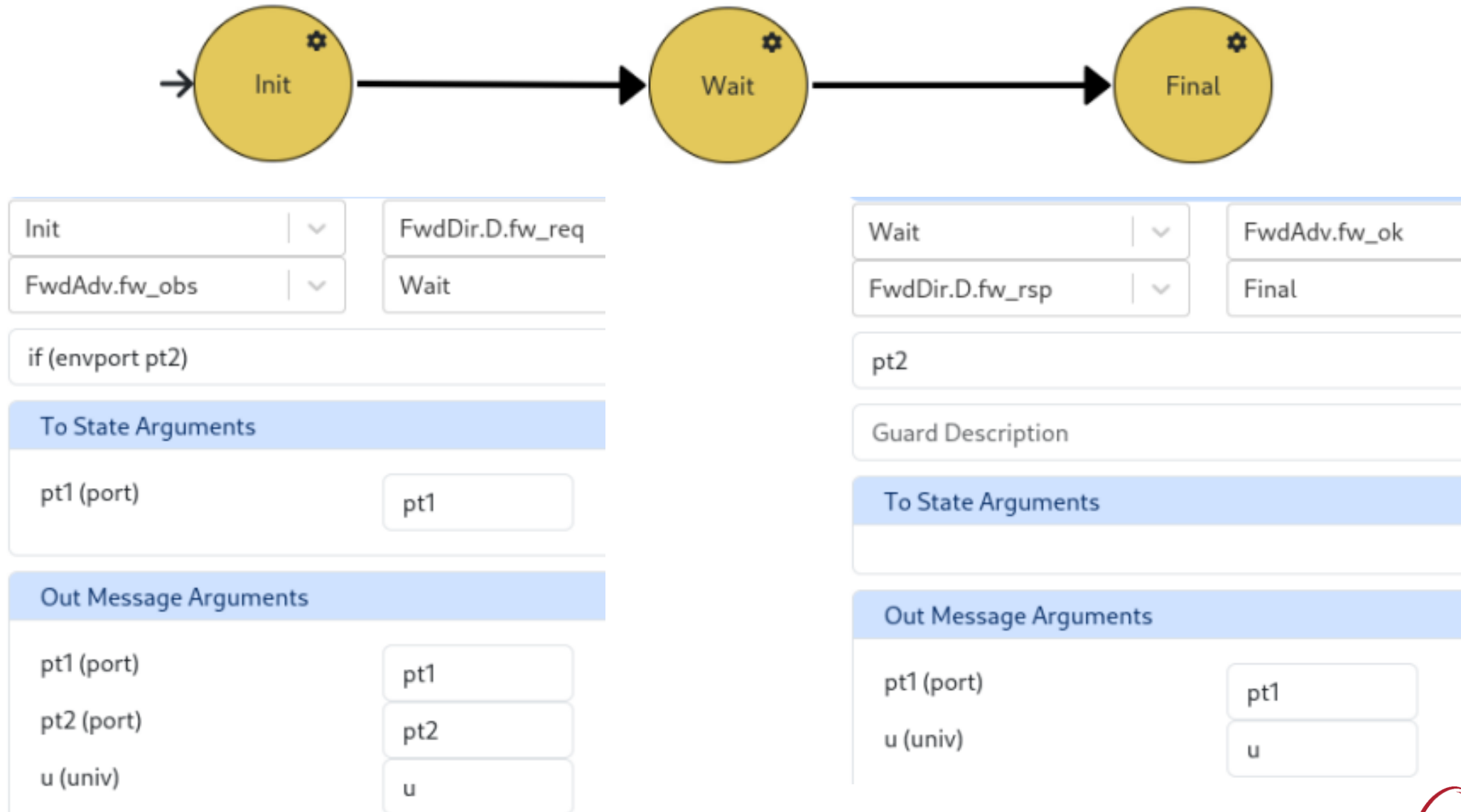
```
INITIATOR(a,na) =  
  user.a?b -> I_running.a.b ->  
  comm!Msg1.a.b.Encrypt.key(b).na.a ->  
  comm.Msg2.b.a.Encrypt.key(a)?na'.nb ->  
  if na==na' then  
    comm!Msg3.a.b.Encrypt.key(b).nb ->  
    I_commit.a.b -> session.a.b -> SKIP  
  else  
    STOP
```

```
RESPONDER(b,nb) =  
  comm.Msg1?a!b.Encrypt.key(b)?na.a' ->  
  if a==a' then  
    R_running.A.B ->  
    comm!Msg2.b.a.Encrypt.key(a).na.nb ->  
    comm.Msg3.a.b.Encrypt.key(b)?nb' ->  
    if nb==nb' then  
      R_commit.a.b -> session.a.b -> SKIP  
    else STOP  
  else STOP
```

- A “specification” of authentication defines processes
 AUTH_INIT = **I_running.A.B** -> **R_commit.A.B** -> AUTH_INIT
 AUTH_RESP = **R_running.A.B** -> **I_commit.A.B** -> AUTH_RESP
- The FDR model checker finds a trace that violates AUTH_INIT

THE MODELS

IDEAL FUNCTIONALITY FOR AN UNTRUSTED NETWORK



IDEAL FUNCTIONALITY FOR AN UNTRUSTED NETWORK (FORWARDING.UC)

```
direct FwDir' {
  in  pt1@fw_req (pt2 : port, u : univ).
  out fw_rsp (pt1 : port, u : univ)@pt2.
}
direct FwDir { D : FwDir' }
adversarial FwAdv {
  out fw_obs (pt1 : port, pt2 : port, u : univ)
  in  fw_ok (pt2 : port, u : univ)
}
functionality Forw implements FwDir FwAdv {
  initial state Init {
    match Message with
    | pt1@FwDir.D.fw_req (pt2, u) => {
      send FwAdv.fw_obs (pt1, pt2, u) and transition Wait (pt1)
    }
  }
  end
}
```

IDEAL FUNCTIONALITY FOR AN UNTRUSTED NETWORK, CONT.

```
state Wait (pt1 : port) {  
  match Message with  
  | FwAdv.fw_ok (pt2, u) => {  
    send FwDir.D.fw_rsp (pt1, u)@pt2  
    and transition Final.  
  }  
  | * => { fail. }  
}  
  
state Final {  
  match Message with  
  | * => { fail. }  
}  
}
```

EASYCRYPT “SUPPORT THEORY” FOR PUBLIC-KEY ENCRYPTION

- **PKE.ec**

```
type pk_t.          (* public keys *)
type sk_t.          (* secret keys *)
type ptxt_t.        (* plain text *)
type ctxt_t = ptxt_t. (* cipher text/signature *)
```

```
op enc (pk: pk_t, p: ptxt_t): ctxt_t.
op dec (sk: sk_t, c: ctxt_t): ptxt_t.
op gen_pair : pk_t -> sk_t -> bool.
```

```
axiom pk_enc_dec (sk : sk_t) (pk : pk_t) (p : ptxt_t) :
  gen_pair pk sk => dec sk (enc pk p) = p.
axiom pk_dec_enc (sk : sk_t) (pk : pk_t) (c : ctxt_t) :
  gen_pair pk sk => enc pk (dec sk c) = c.
```

```
hint simplify pk_enc_dec, pk_dec_enc.
```


EASYPHYPT “SUPPORT THEORY” FOR PUBLIC-KEY ENCRYPTION, CONT.

- PKE_EPDP.ec

```
require import PKE UCUniv.  
(*---*) import UCEncoding.
```

EPDP = Encoding and Partial Decoding Pair

```
type ('a, 'b) epdp = {  
  enc : 'a -> 'b;  
  dec : 'b -> 'a option  
}.
```

```
op [opaque smt_opaque] epdp_cipher_univ : (ctxt_t, univ) epdp.  
axiom valid_epdp_cipher_univ : valid_epdp epdp_cipher_univ.  
hint simplify valid_epdp_cipher_univ.
```

```
op [opaque smt_opaque]  
  epdp_plain_pair_plain : (ptxt_t * ptxt_t, ptxt_t) epdp.  
axiom valid_epdp_plain_pair_plain : valid_epdp epdp_plain_pair_plain.  
hint simplify valid valid_epdp_plain_pair_plain.
```

EASYCRYPT SUPPORT THEORY FOR NEEDHAM-SCHROEDER

- **NeedhamSchroeder.ec**

```
require import Distr Int PKE PKE_EPDP UCBasicTypes.
```

```
op dnonce : int distr = drange 0 184467440730951616. (* 2^64 *)
```

```
const pk_a, pk_b : pk_t.
```

```
const sk_a, sk_b : sk_t.
```

```
axiom gp_pk_sk_a : gen_pair pk_a sk_a.
```

```
axiom gp_pk_sk_b : gen_pair pk_b sk_b.
```

```
hint simplify gp_pk_sk_a, gp_pk_sk_b.
```

```
op [opaque smt_opaque] epdp_port_port_cipher_univ :
```

```
  (port * port * ctxt_t, univ) epdp =
```

```
  epdp_tuple3_univ epdp_port_univ epdp_port_univ epdp_cipher_univ.
```

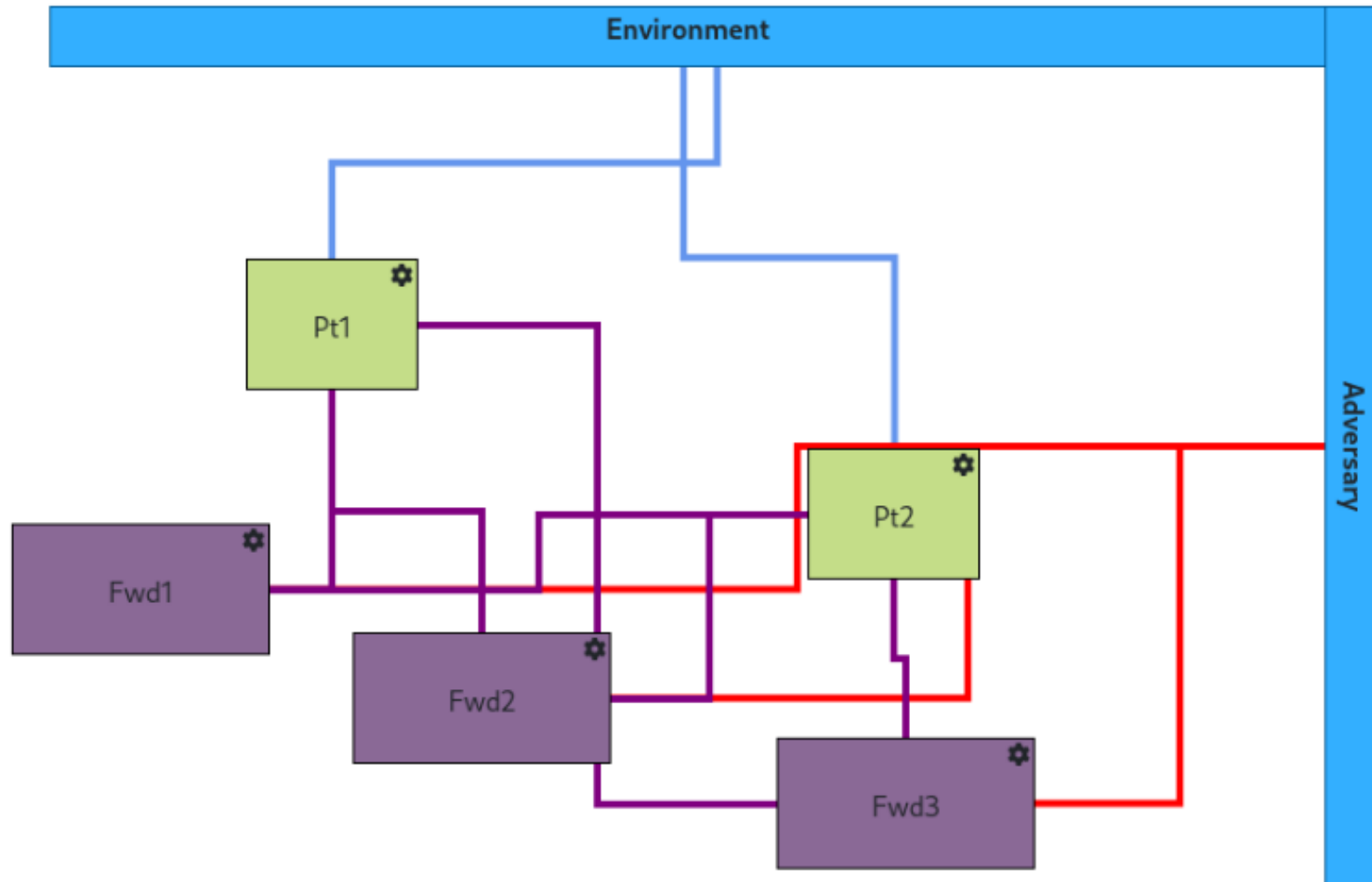
```
lemma valid_epdp_port_port_cipher_univ :
```

```
  valid_epdp epdp_port_port_cipher_univ
```

```
  by rewrite /epdp_port_port_cipher_univ.
```

THE REAL WORLD FOR NEEDHAM-SCHROEDER

NSReal



INTERFACE DEFINITIONS

Composite Interfaces

NSDir ^

NSDir Direct Adversarial

Basic Interfaces +

Pt1D Pt1Dir | v

Pt2D Pt2Dir | v

Basic Interfaces

Pt1Dir ^

Pt1Dir Direct Adversarial

Messages +

ns_req v

Pt2Dir ^

Pt2Dir Direct Adversarial

Messages +

ns_acc v

NSI2S ^

NSI2S Direct Adversarial

Messages +

leak v

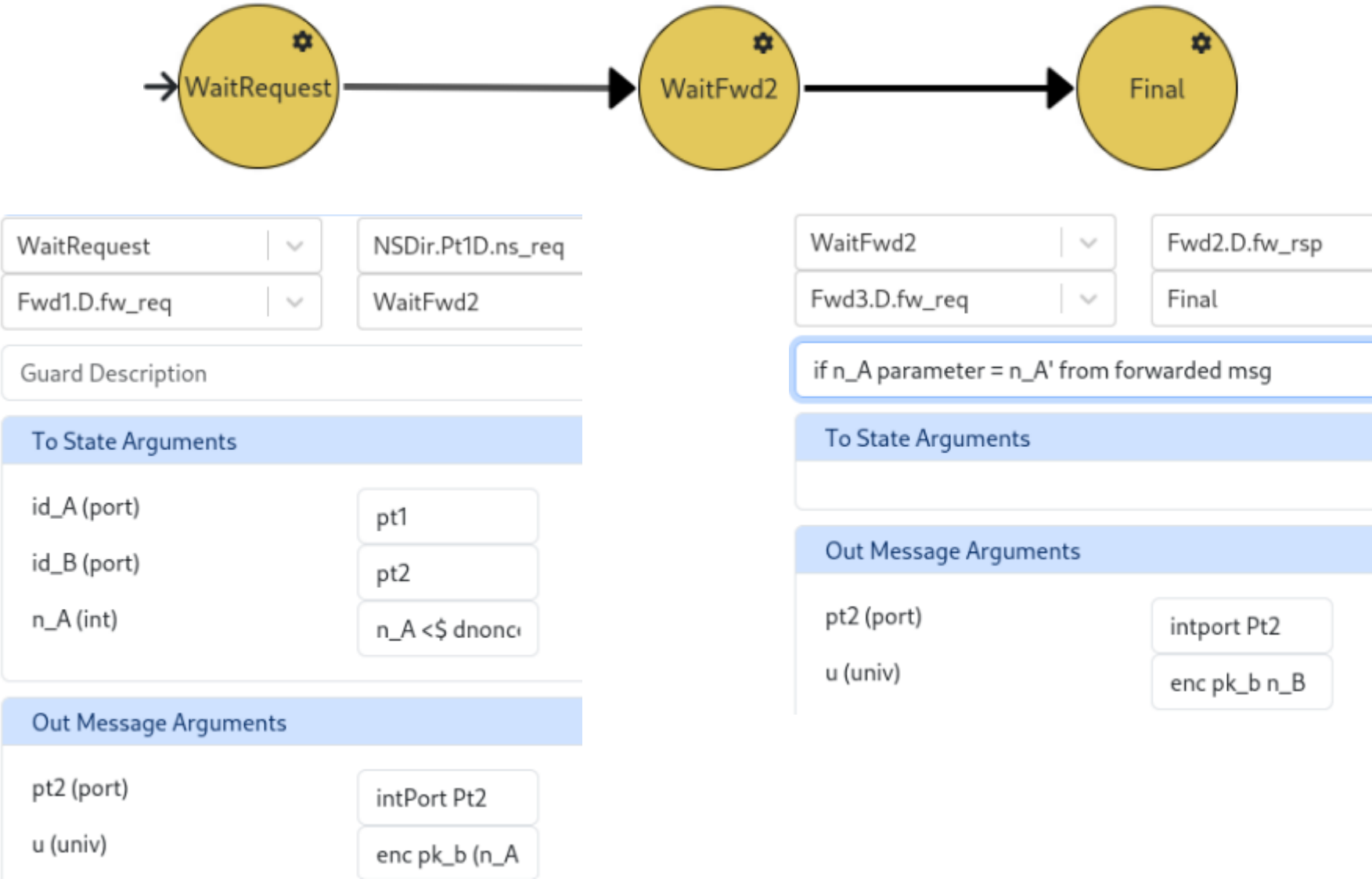
ok v

in pt1@ns_req (pt2 : port)

out ns_acc (pt1 : port)@pt2

out leak (pt1 : port, pt2 : port)
in ok

PARTY PT1 STATE MACHINE



PARTY PT2 STATE MACHINE



WaitFwd1	▼	Fwd1.D.fw_rsp
Fwd2.D.fw_req	▼	WaitFwd3

Guard Description

To State Arguments

id_A (port)	id_A from forw
id_B (port)	id_B from forw
n_A (int)	n_A from forw
n_B (int)	n_B <\$ dnonce

Out Message Arguments

pt2 (port)	intport Pt1
u (univ)	enc pk_b (n_A

WaitFwd3	▼	Fwd3.D.fw_rsp
NSDir.Pt2D.ns_acc	▼	Final

pt2

if n_B parameter = n_B' from forwarded msg

To State Arguments

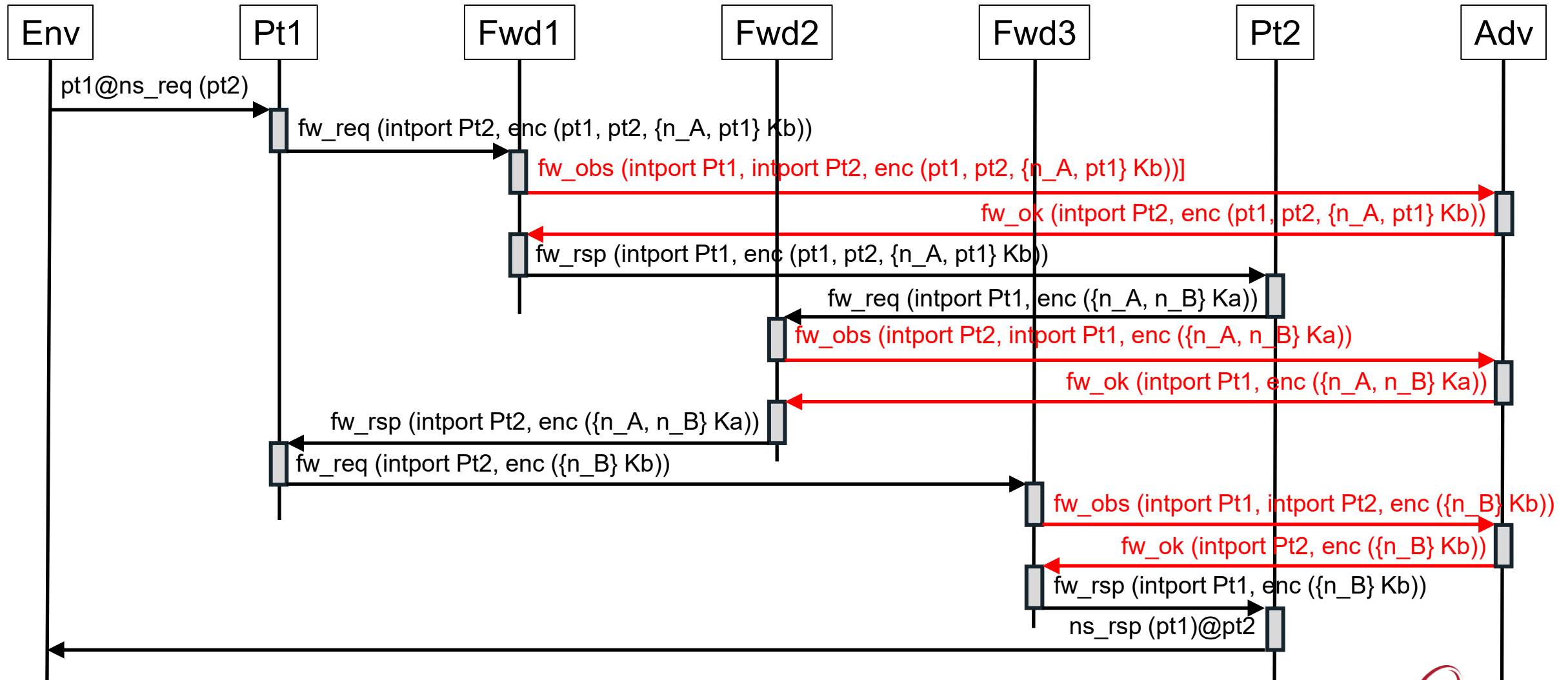
Out Message Arguments

pt1 (port)	id_A
------------	------

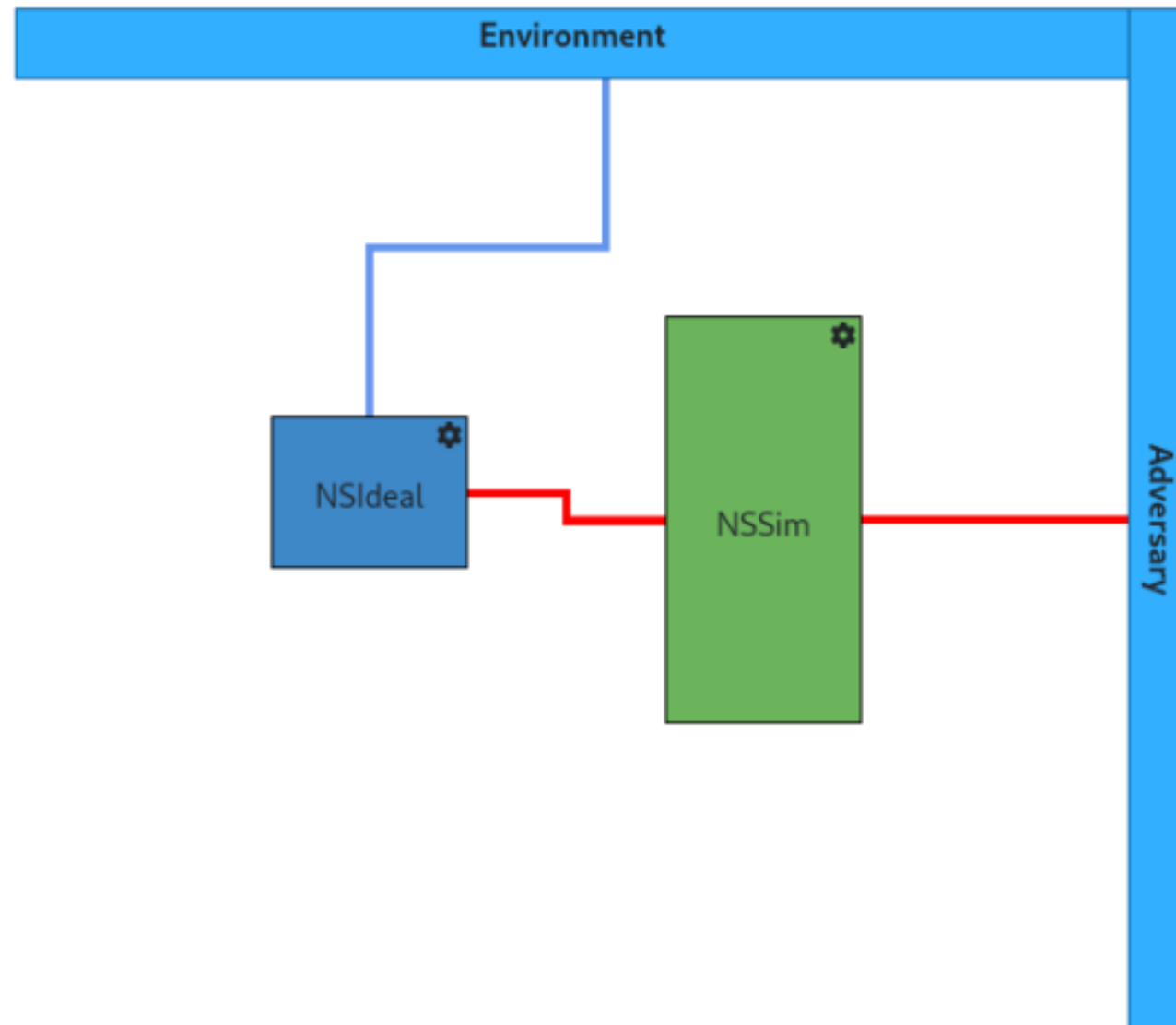
SAMPLE DSL CODE FROM PARTY PT2

```
state WaitFwd3 (id_A : port, id_B : port, n_A : int, n_B : int) {
  match message with
  | Fwd3.D.fw_rsp (_, u) => {
    match epdp_cipher_univ.`dec u with
    | Some ciphertext => {
      match epdp_int_plain.`dec (dec sk_b ciphertext) with
      | Some n_B' => {
        if (n_B' <> n_B) { fail. }
        else {
          send NSDir.Pt2D.ns_acc (id_A)@id_B
          and transition Final.
        }
      }
      | None => { fail. }
    end
  }
  | None => { fail. }
end
}
```

REAL WORLD SEQUENCE DIAGRAM



THE IDEAL WORLD FOR NEEDHAM-SCHROEDER



NSIDEAL STATE MACHINE



WaitRequest	▼	NSDir.Pt1D.ns_req
NSI2S.leak	▼	WaitSim

Guard Description

To State Arguments

id_A (port)

pt1

id_B (port)

pt2

Out Message Arguments

pt1 (port)

pt1

pt2 (port)

pt2

WaitSim	▼	NSI2S.ok
NSDir.Pt2D.ns_acc	▼	Final

id_B

Guard Description

To State Arguments

Out Message Arguments

pt1 (port)

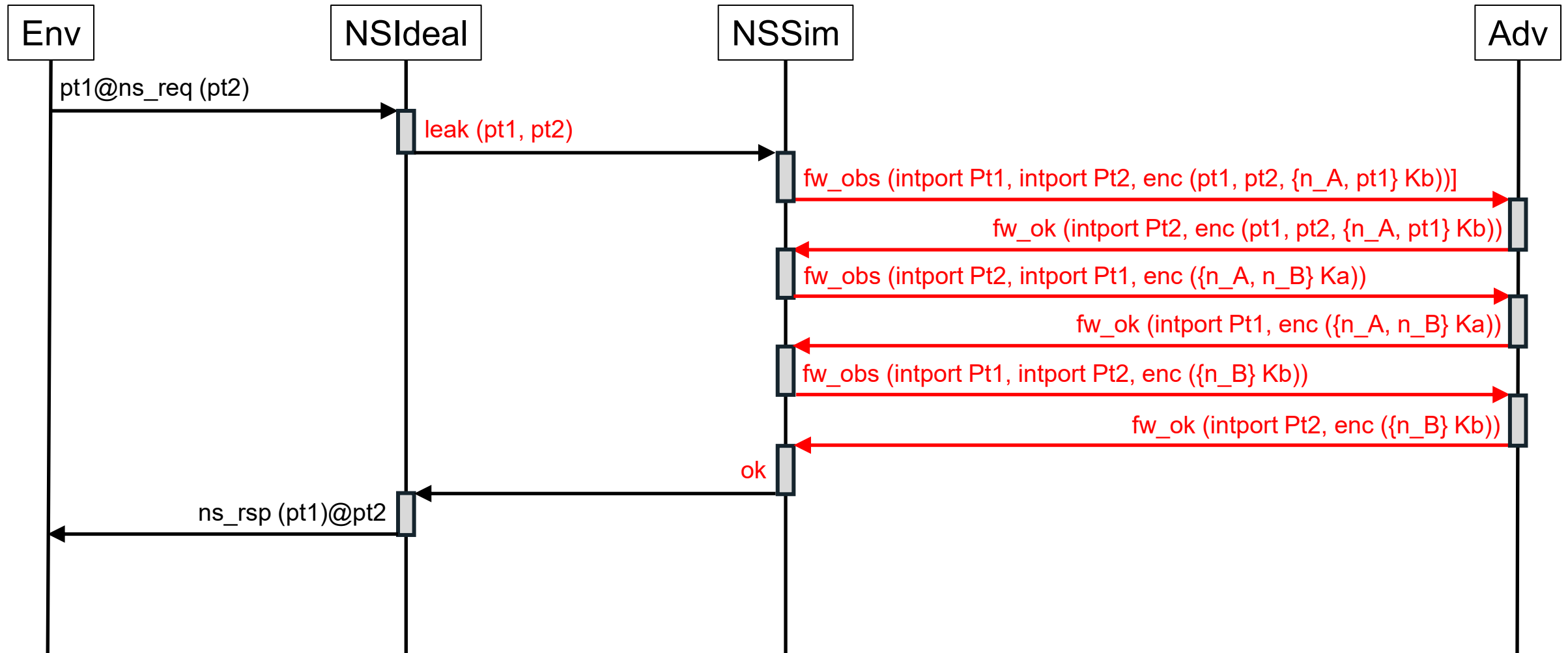
id_A

NSSIM STATE MACHINE



WaitIdeal	NSI2S.leak	WaitAdv1	Fwd1.FwdAdv.fw_ok	WaitAdv2	Fwd2.FwdAdv.fw_ok	WaitAdv3	Fwd3.FwdAdv.fw_ok
Fwd1.FwdAdv.fw_obs	WaitAdv1	Fwd2.FwdAdv.fw_obs	WaitAdv2	Fwd3.FwdAdv.fw_obs	WaitAdv3	NSI2S.ok	Final
Guard Description		if id_A param = id_A' from msg and id_B param		if n_A param = n_A' from msg and id_B param		if n_B param = n_B' from msg	
To State Arguments		To State Arguments		To State Arguments		To State Arguments	
id_A (port)	pt1	n_A (int)	n_A	n_B (int)	n_A	Out Message Arguments	
id_B (port)	pt2	n_B (int)	n_B <\$ dnonce				
n_A (int)	n_A <\$ dnonce	Out Message Arguments		Out Message Arguments			
Out Message Arguments		Out Message Arguments		Out Message Arguments			
pt1 (port)	intport Pt1	pt1 (port)	intport Pt2	pt1 (port)	intport Pt1		
pt2 (port)	intport Pt2	pt2 (port)	intport Pt1	pt2 (port)	intport Pt2		
u (univ)	enc pk_b (n_A	u (univ)	enc pk_a (n_A	u (univ)	enc pk_b n_B		

IDEAL WORLD SEQUENCE DIAGRAM



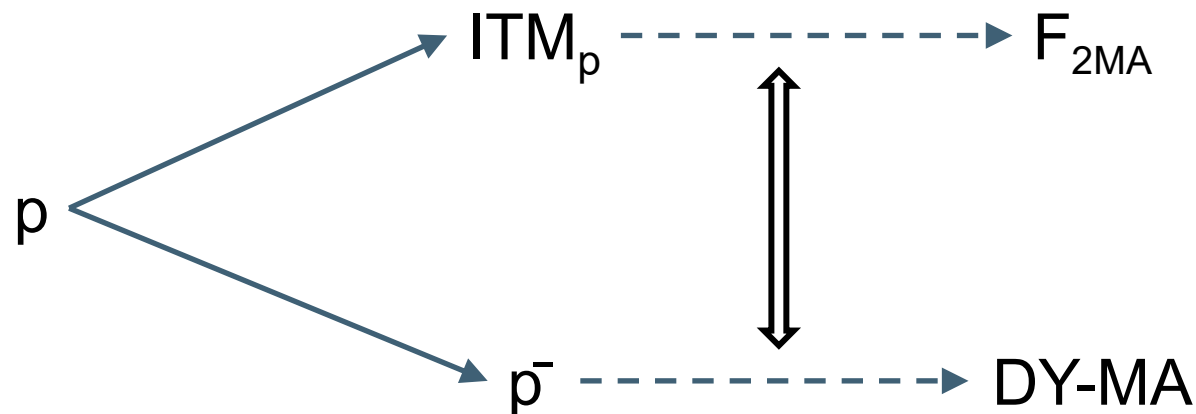
DISCUSSION

ANALYSIS OF THE EASYUC MODEL

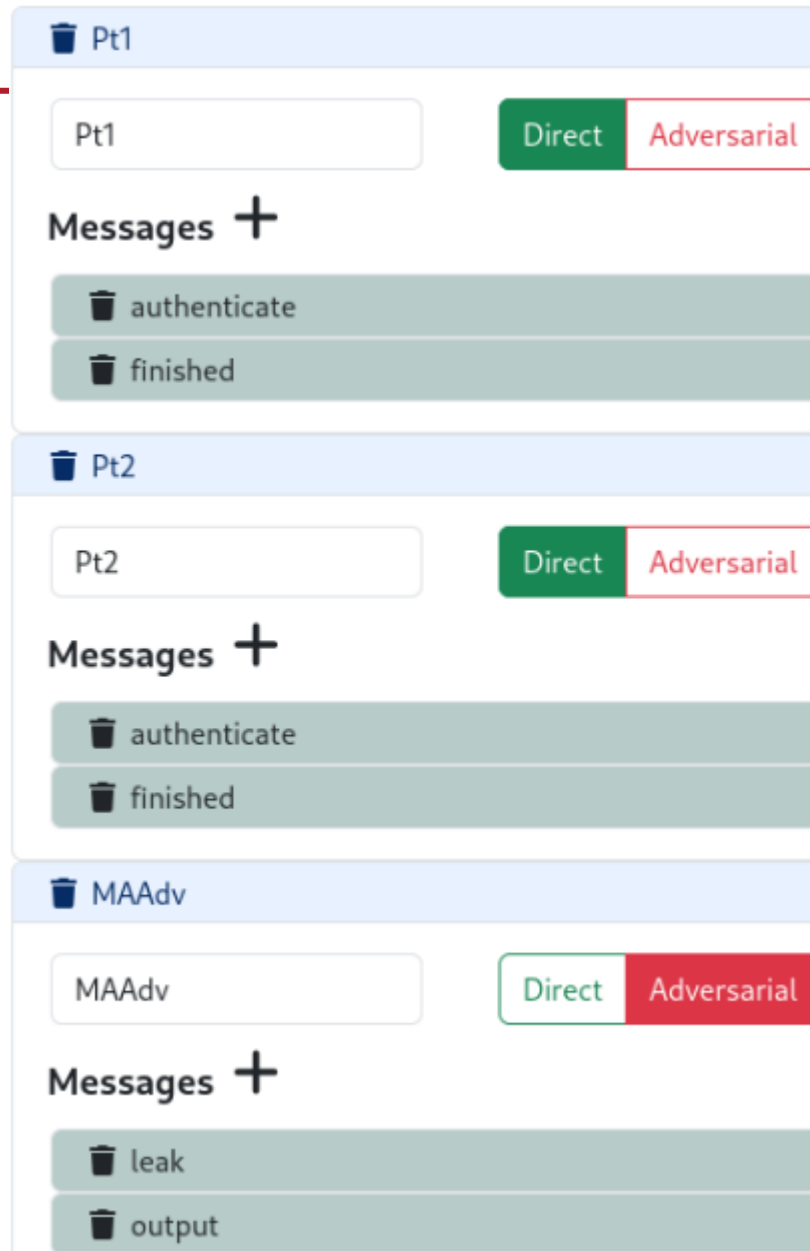
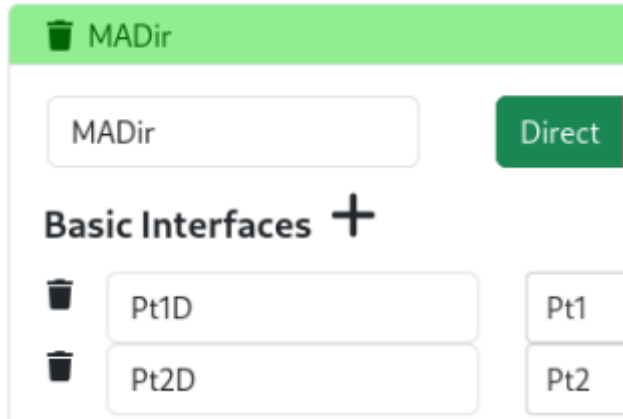
- What does the ideal functionality guarantee?
 - B authenticates A
 - A authenticates B
 - What definition of authentication is this?
- How would A and B carry on a conversation?
 - Maybe they should swap nonces...

SYMBOLIC ANALYSIS OF UC MODELS

- Canetti and Herzog, “Universally Composable Symbolic Security Analysis,” 2011
 - The Dolev-Yao model for symbolic encryption
 - Message algebra, symbolic protocols, adversary and executions
 - *Simple protocols* with 2 principals
 - Symbolic and UC semantics in terms of constructions
 - Symbolic analysis of UC mutual authentication
 - Theorem 14. A simple protocol p UC-realizes F_{2MA} iff the corresponding Dolev-Yao model p^- satisfies the symbolic criterion DY-MA
 - Informally, if A completes a session with B, then B has started a session with A



F_{2MA} IN EASYUC

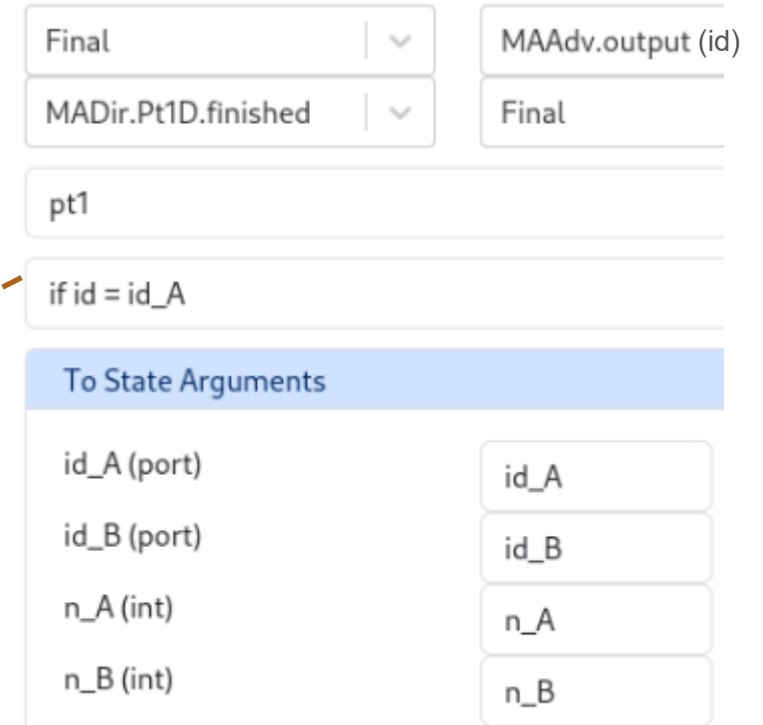
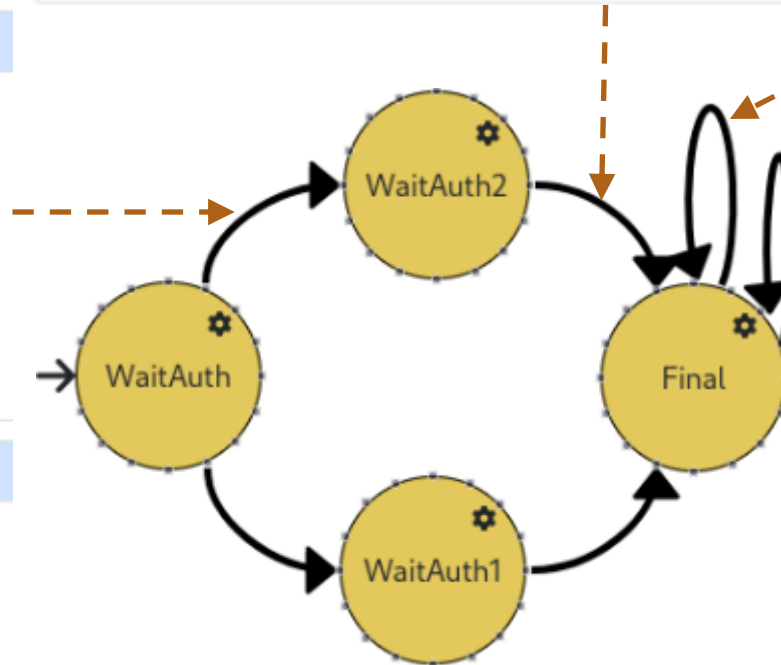
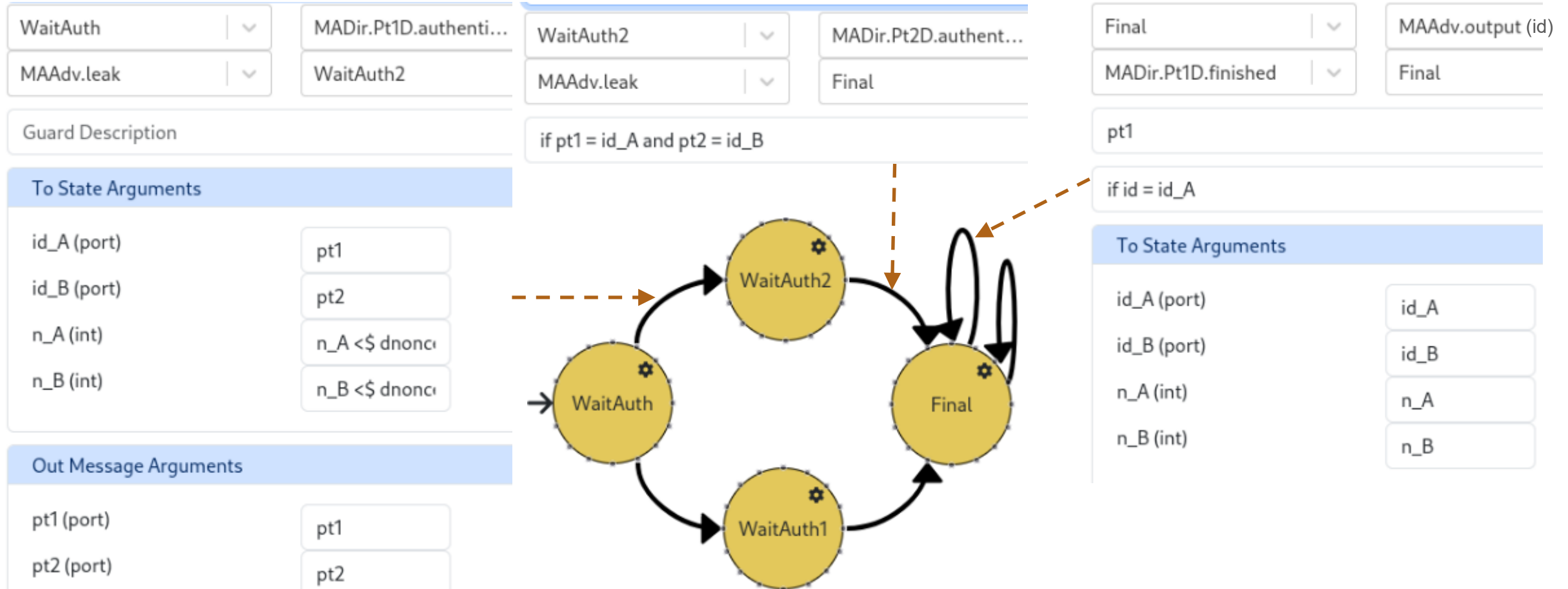


in pt1@authenticate (pt2 : port)
out finished@pt1

in pt2@authenticate (pt1 : port)
out finished@pt2

out leak (pt1 : port, pt2 : port))
in finished (pt : port)

F_{2MA} IN EASYUC, CONT



F_{KE} IN EASYUC

 KEDir

KEDir

Direct Adversarial

Basic Interfaces +

 Pt1

KEDirPt1

 Pt2

KEDirPt2

 KEI2S

KEI2S

Direct Adversarial

Messages +

 ke_sim_req1

out ke_sim_req1 (pt1 : port, pt2 : port))

 ke_sim_rsp

in ke_sim_rsp

 ke_sim_req2

out ke_sim_req2

 KEDirPt1

KEDirPt1

Direct Adversarial

Messages +

 ke_req1

in pt1@ke_req1 (pt2 : port)

 ke_rsp2

out ke_rsp2 (k : Key)@pt1

 KEDirPt2

KEDirPt2

Direct Adversarial

Messages +

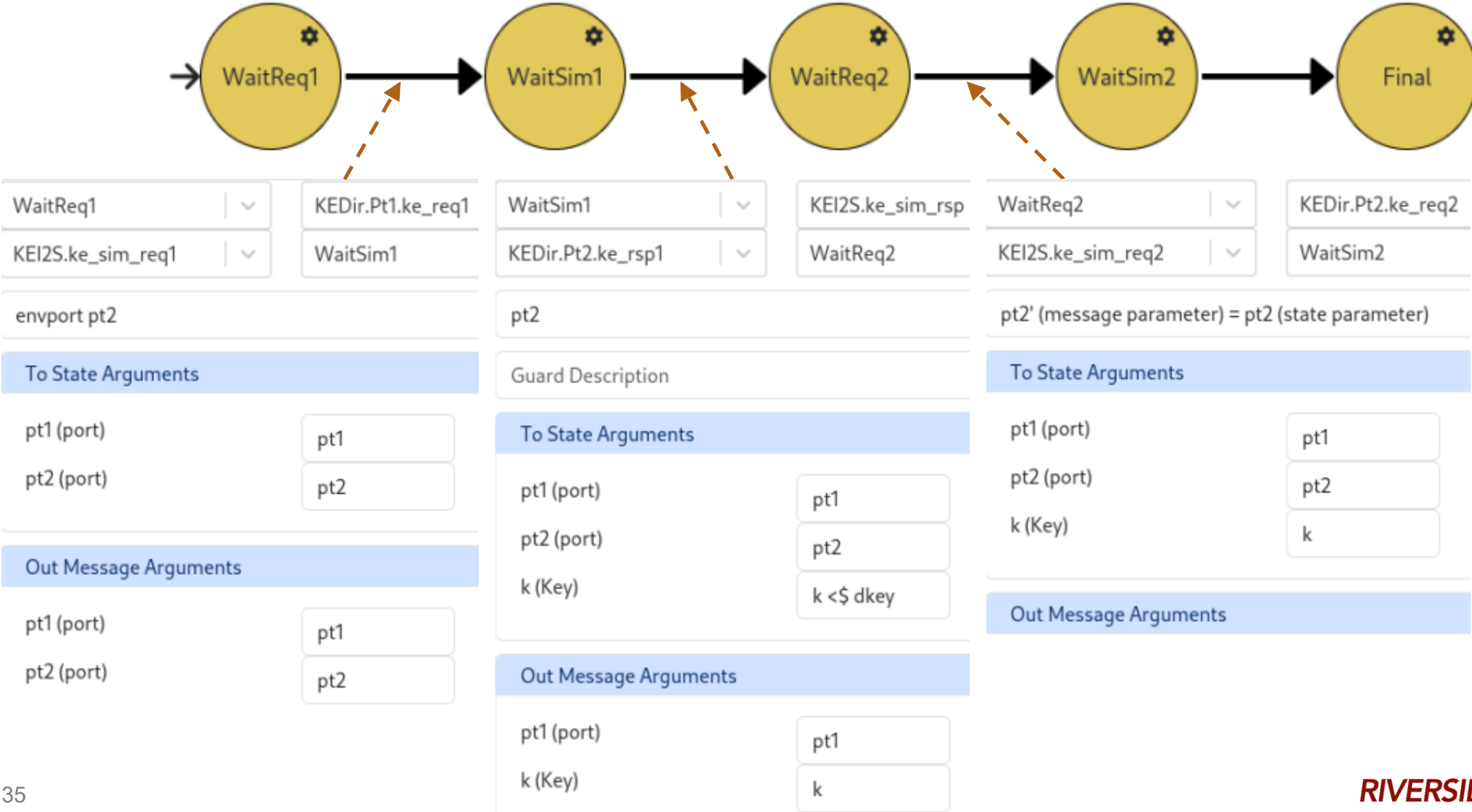
 ke_req2

in pt2@ke_req2 (pt1 : port)

 ke_rsp1

out ke_rsp1 (k : Key)@pt2

F_{KE} IN EASYUC, CONT.



REVISED INTERFACES FOR NEEDHAM-SCHROEDER

NSNEDir

NSNEDir

Direct

Adversarial

Basic Interfaces +



Pt1D

Pt1Dir



Pt2D

Pt2Dir

NSNEI2S

NSNEI2S

Direct

Adversarial

Messages +



leak

out leak (pt1 : port, pt2 : port)



ok

in ok)

Pt1Dir

Pt1Dir

Direct

Adversarial

Messages +



ns_req

in pt1@ns_req (pt2 : port)



ns_acc

out ns_acc (n_A : int, n_B : int)@pt1



ns_ack

in pt1@ns_ack

Pt2Dir

Pt2Dir

Direct

Adversarial

Messages +



ns_req

out ns_req (pt1 : port)@pt2



ns_acc

in pt2@ns_acc



ns_ack

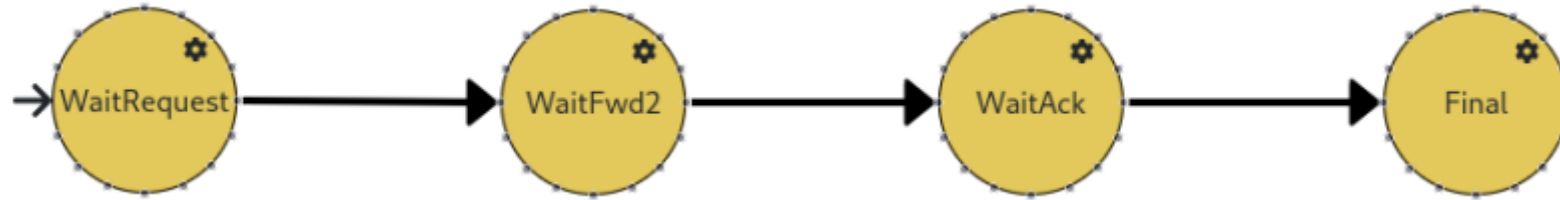
out @ns_ack (n_A : int, n_B : int)

REVISED IDEAL FUNCTIONALITY FOR NEEDHAM-SCHROEDER



REVISED REAL FUNCTIONALITY FOR NEEDHAM-SCHROEDER

- Party Pt1 state machine



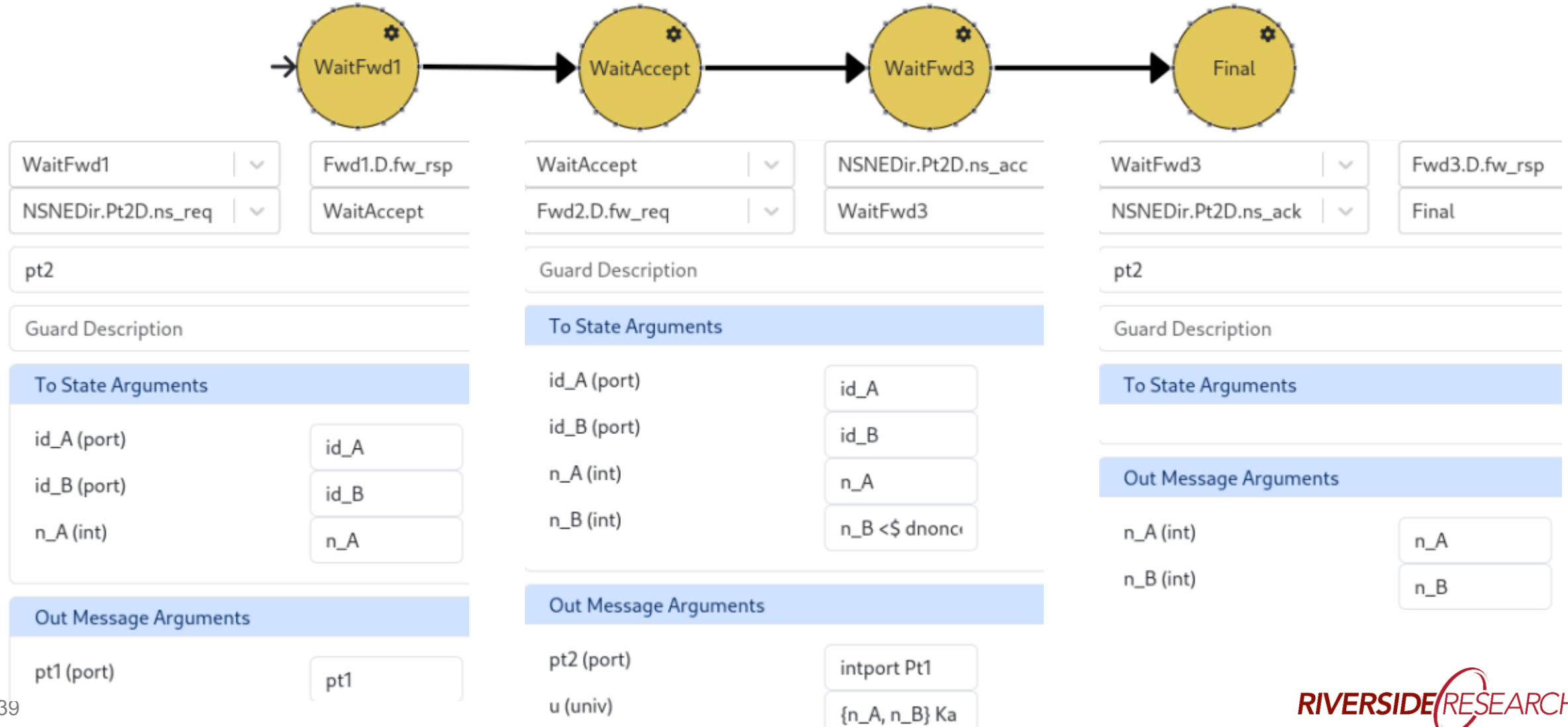
WaitRequest	▼	NSNEDir.Pt1D.ns_req
Fwd1.D.fw_req	▼	WaitFwd2
Guard Description		
To State Arguments		
id_A (port)	pt1	
id_B (port)	pt2	
n_A (int)	n_A <\$ dnonce	
Out Message Arguments		
pt2 (port)	intport Pt2	
u (univ)	pt1, pt2, {n_A,	

Enter a name		
WaitFwd2	▼	Fwd2.D.fw_rsp
NSNEDir.Pt1D.ns_acc	▼	WaitAck
pt1		
if n_A = n_A' from u		
To State Arguments		
Out Message Arguments		
n_A (int)	n_A	
n_B (int)	n_B	

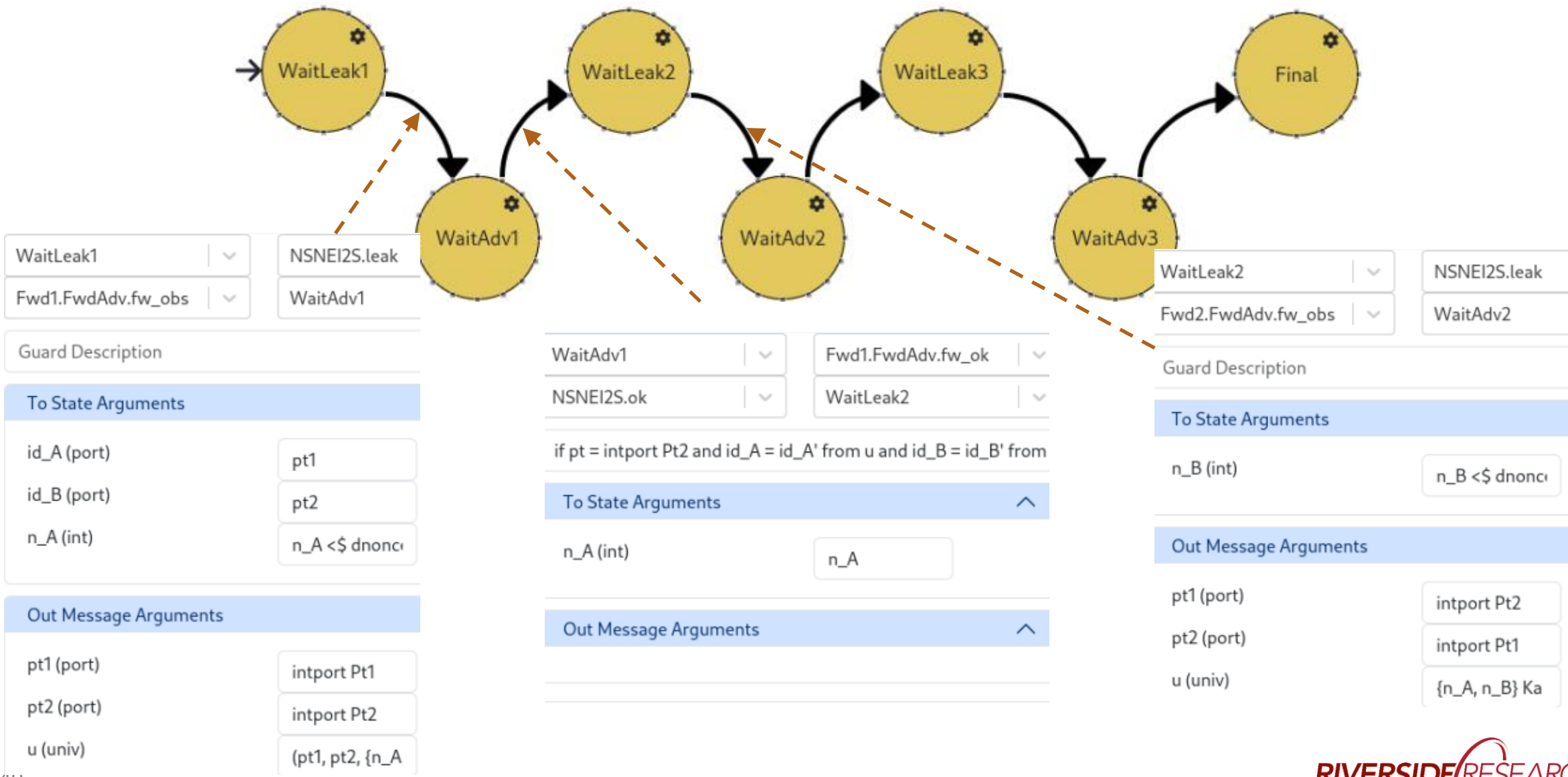
WaitAck	▼	NSNEDir.Pt1D.ns_ack
Fwd3.D.fw_req	▼	Final
if pt1 = id_A		
To State Arguments		
Out Message Arguments		
pt2 (port)	intport Pt2	
u (univ)	{n_B} Kb	

REVISED REAL FUNCTIONALITY FOR NEEDHAM-SCHROEDER

- Party Pt2 state machine



REVISED SIMULATOR FOR NEEDHAM-SCHROEDER



MODELING THE MITM ATTACK

APPROACH

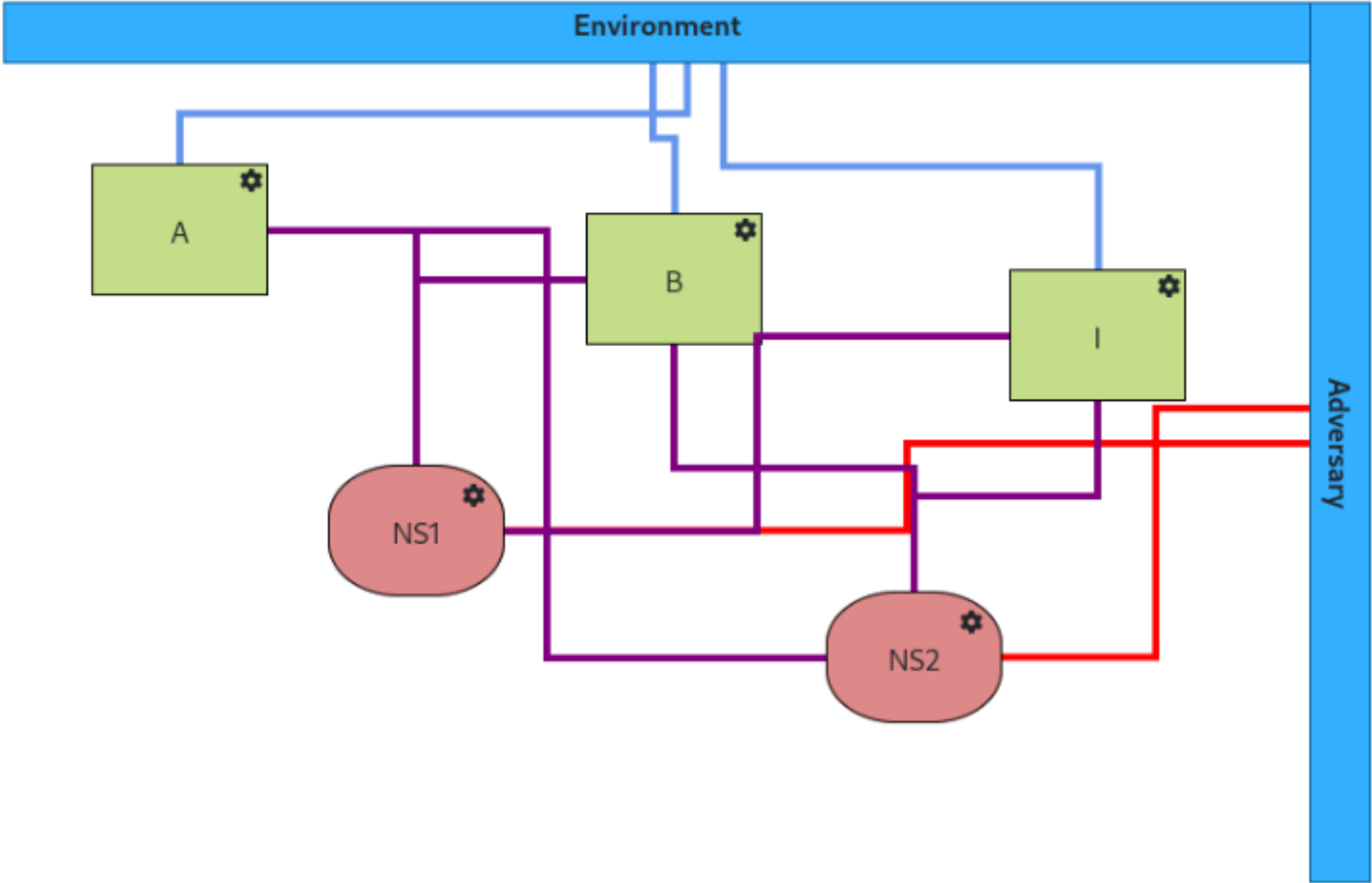
- In NSReal, replace hard-coded keys with keys provided by `init_PKE` messages:

```
in  pt1@init_PKE (sk_a : sk_t, pk_table : (port pk_t) fmap)
out init_PKE_resp@pt1
```

 - NSIdeal will generate the response but otherwise ignore the message
- Create a higher-level model containing two instances of Needham-Schroeder and 3 parties:
 - Party A is initiator of NS1; Party B is responder of NS2; Party I is responder of NS1 and initiator of NS2
- Option 1 (chosen)
 - NS1 and NS2 behave normally but party I has shared its key with the adversary
 - The adversary intercepts and modifies network traffic through forwarders (as already allowed)
- Option 2
 - Modify Needham-Schroeder to allow for corruption of a party
 - I.e., the adversary tells it what to send at each step
 - No need for adversary to modify any network traffic
- The ideal functionality and simulator are vestigial; just enough to satisfy the type checker

MITM ATTACK REAL WORLD

NSMITMReal



MITM ATTACK INTERFACES

Composite Interfaces

NSMITMDir

NSMITMDir

Direct

Adversary

Interface Comment

Basic Interfaces +

PtAD

PtADir

PtBD

PtBDir

PtID

PtIDir

Basic Interfaces

PtADir

PtADir

Direct

Adversary

Interface for the principal A

Messages +

init_PKE

part1

ok

PtBDir

PtBDir

Direct

Adversary

Interface for the principal B

Messages +

init_PKE

ok

PtIDir

PtIDir

Direct

Adversary

Interface for the principal I, for Intruder (the adversary)

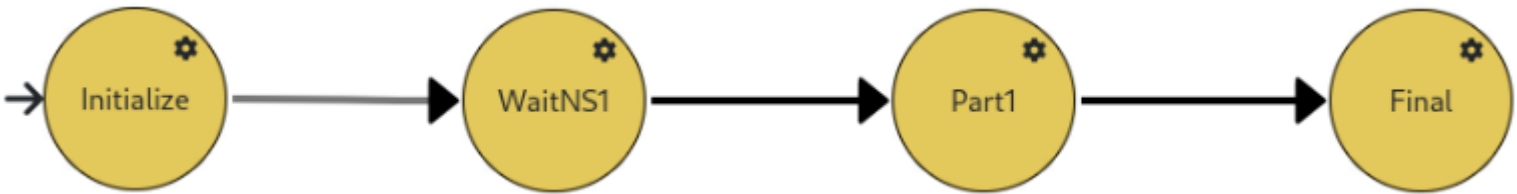
Messages +

init_PKE

part2

ok

PARTY A STATE DIAGRAM



Initialize	NSMITMDir.PtAD.ini...	WaitNS1	NS1.Pt1D.initPKE_resp	Part1	NSMITMDir.PtAD.pa...
NS1.Pt1D.initPKE	WaitNS1	NSMITMDir.PtAD.ok	Part1	NS1.Pt1D.ns_req	Final
Guard Description		respPort	Guard Description		
To State Arguments		Guard Description	To State Arguments		
respPort (port)	pt	To State Arguments	Out Message Arguments		
Out Message Arguments		Out Message Arguments	pt2 (port)	intport l	
sk_a (sk_t)	sk_a				
pk_table ((port, pk_t) fmap)	pk_table				

PARTY B STATE DIAGRAM



Initialize

NS2.Pt2D.initPKE

NSMITMDir.PtBD.ini...

WaitNS2Init

Guard Description

To State Arguments

respPort (port)

pt

Out Message Arguments

sk_b (sk_t)

sk_B

pk_table ((port, pk_t) fmap)

pk_table

WaitNS2Init

NSMITMDir.PtBD.ok

NS2.Pt2D.initPKE_r...

WaitNS2Accept

respPort

Guard Description

To State Arguments

respPort (port)

respPort

Out Message Arguments

WaitNS2Accept

NSMITMDir.PtBD.ok

NS2.Pt2D.ns_acc

Final

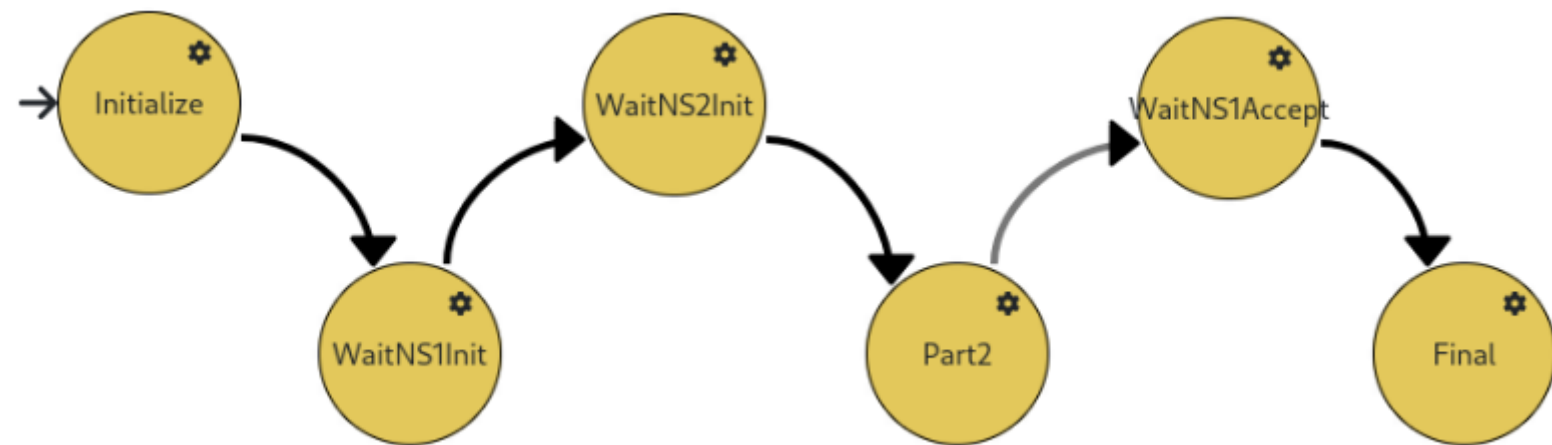
respPort

Guard Description

To State Arguments

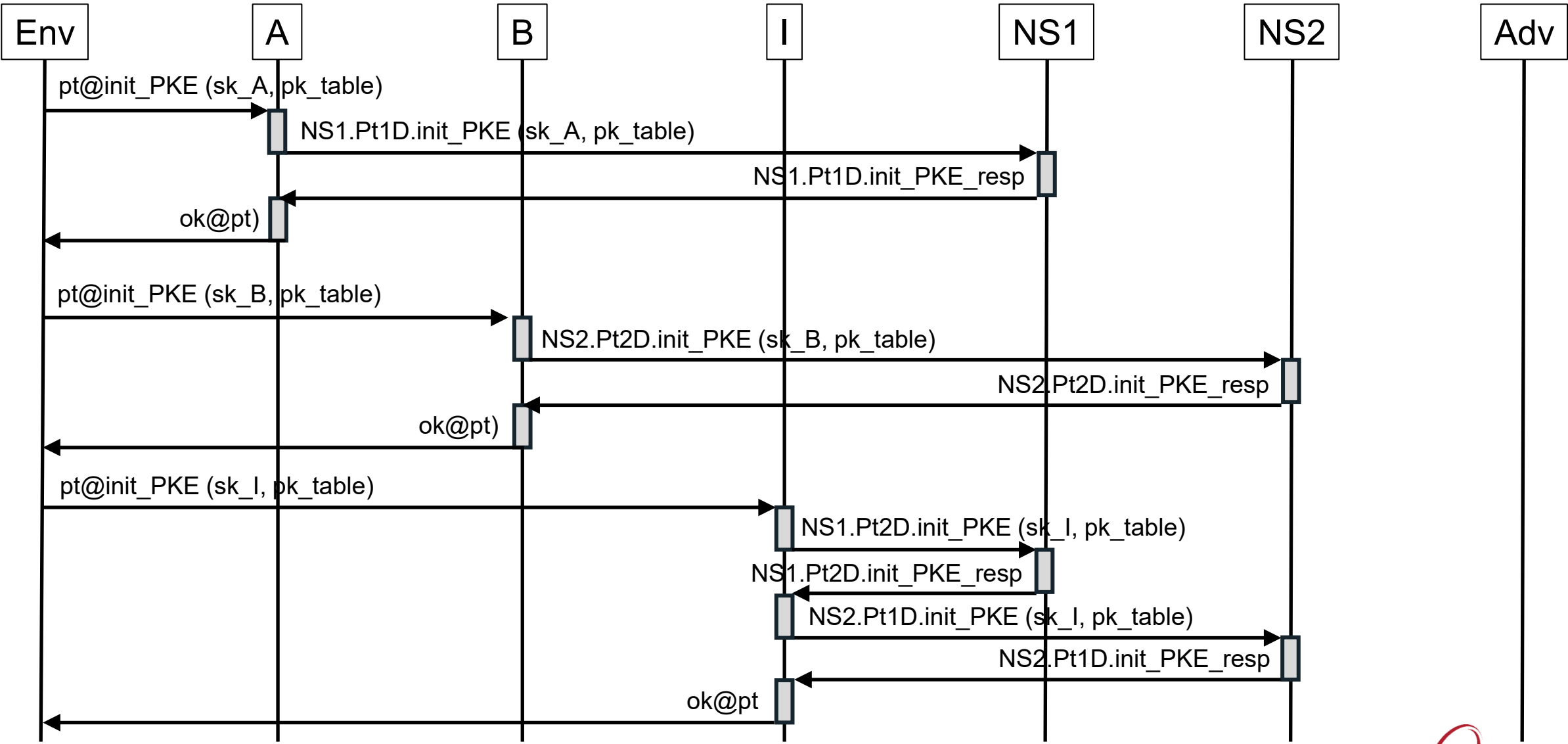
Out Message Arguments

PARTY I STATE DIAGRAM

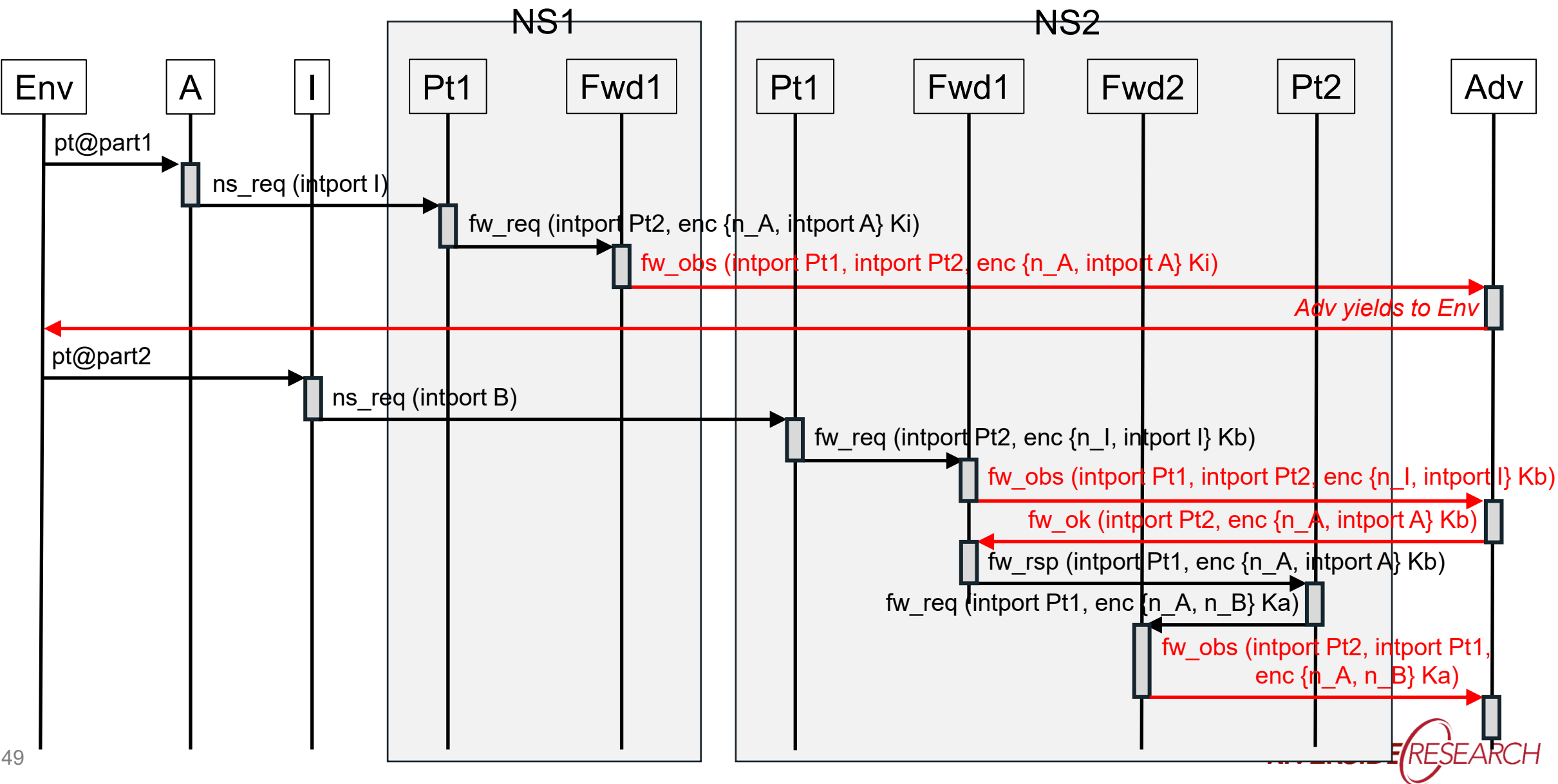


From State	In Message	Out Message	To State
Initialize	pt@NSMITMDir.PtID.initPKE (sk_l, pk_table)	NS1.Pt2D.initPKE (sk_l, pk_table)	WaitNS1Init
WaitNS1Init	NS1.Pt2D.initPKE_resp	NS2.Pt1D.initPKE (sk_l, pk_table)	WaitNS2Init
WaitNS2Init	NS2.Pt1D.initPKE_resp	NSMITMDir.Ptld.ok	Part2
Part2	NSMITMDir.Ptld.part2	NS2.Pt1D.ns_req (intport B)	WaitNS1Accept
WaitNS1Accept	NS1.Pt2D.ns_acc (_)	NSMITMDir.PtID.ok	Final

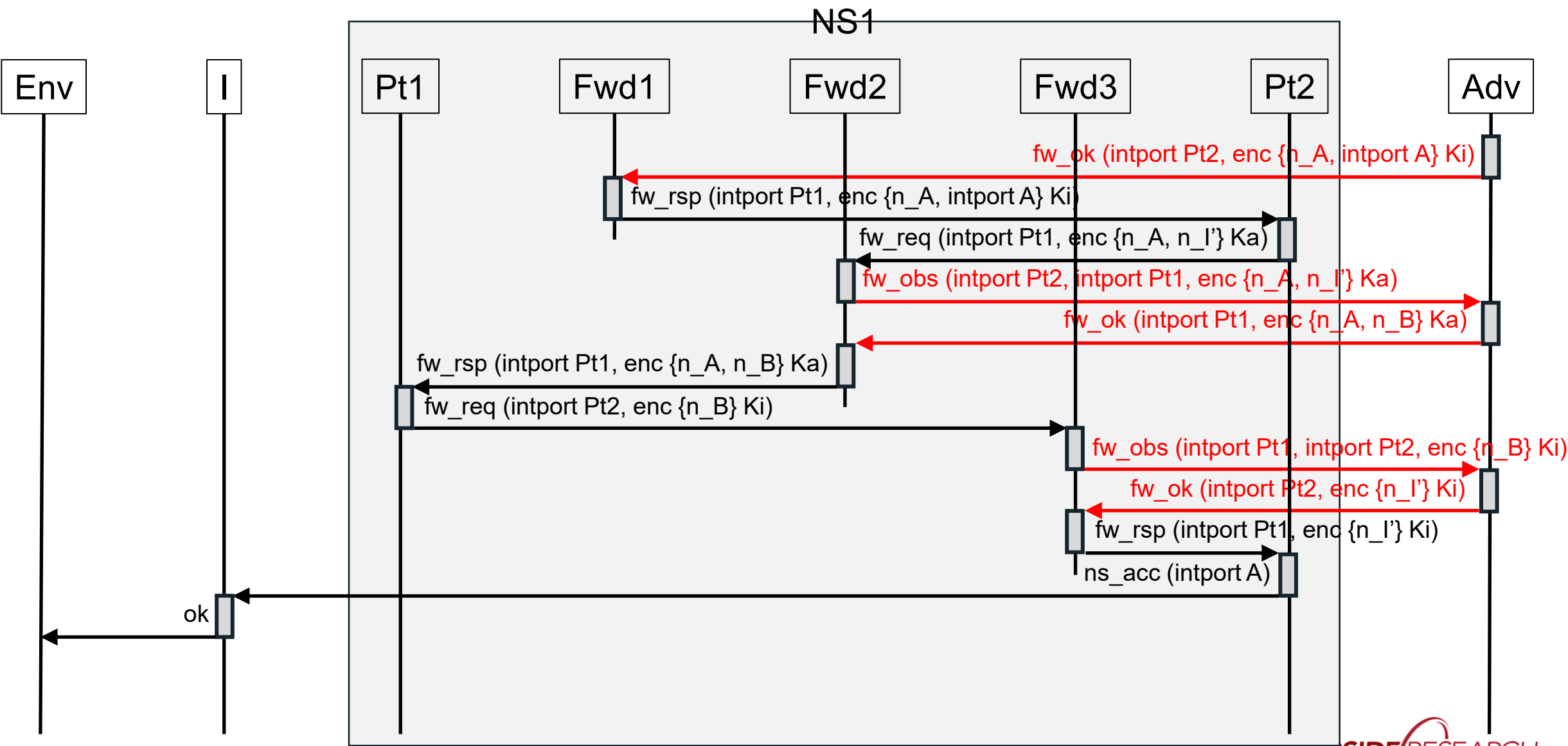
SEQUENCE DIAGRAM OF EXPLOIT (INITIALIZATION)



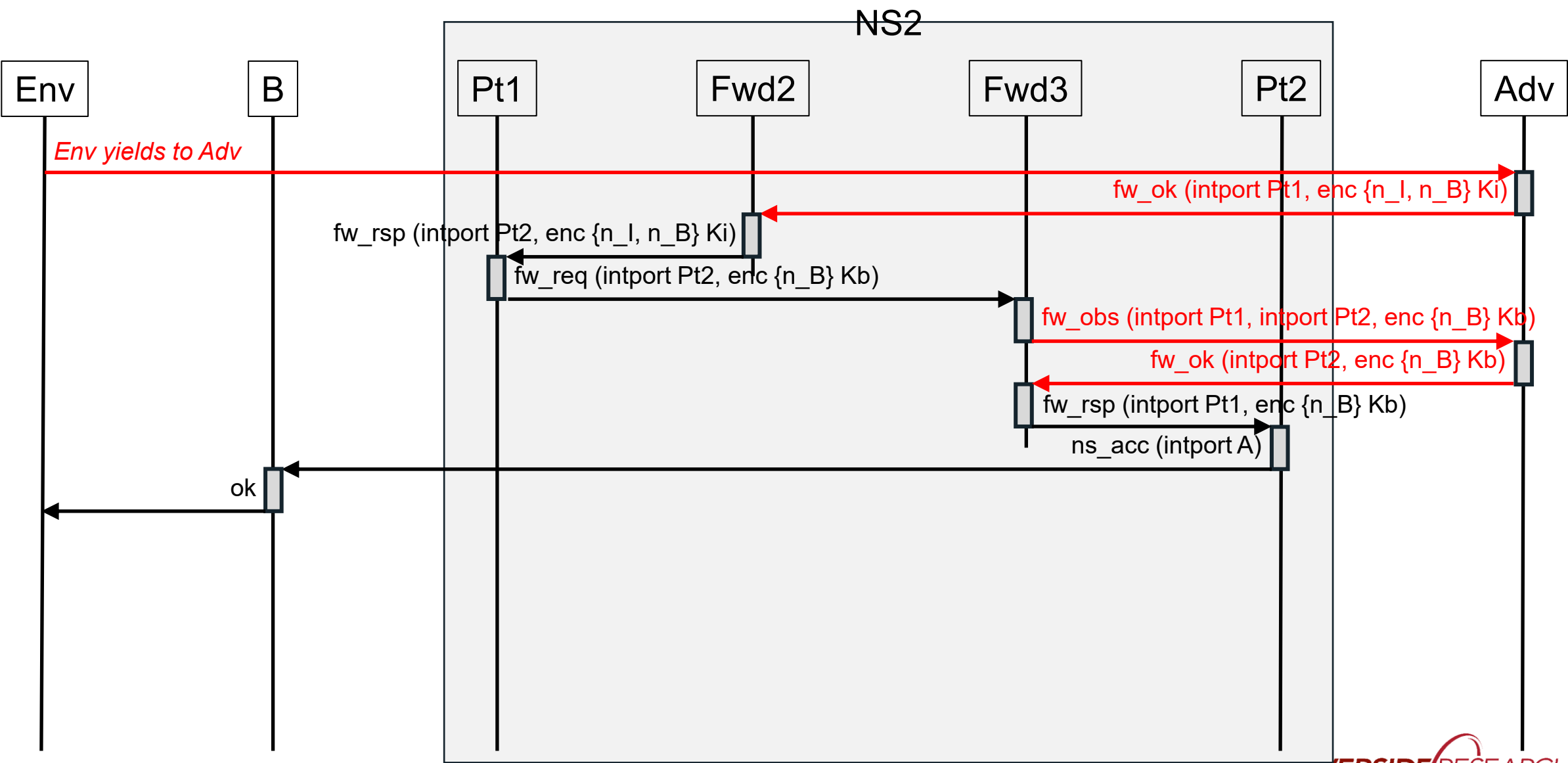
SEQUENCE DIAGRAM OF EXPLOIT, CONT



SEQUENCE DIAGRAM OF EXPLOIT, CONT



SEQUENCE DIAGRAM OF EXPLOIT, CONT



EXPLOIT RESULTS

- NS1: A $\leftrightarrow$ I
 - A thinks the nonces are Na and Nb
 - I thinks the nonces are Na and Ni'
- NS2: I $\leftrightarrow$ B
 - I thinks the nonces are Ni and Nb
 - B thinks the nonces are Na and Nb and the session is with A
- The adversary must continue to intercept and alter messages to maintain the ruse
 - A $\rightarrow$ I: {M, Nb} Ki becomes A $\rightarrow$ B: {M, Nb} Kb (and Adv learns M)
 - B $\rightarrow$ I: {M, Na} Ka becomes B $\rightarrow$ A : {M, Na} Ka (Adv can't read these, though)
 - Principal I is not involved

Not the original exploit, but similar

REFERENCES

- Ross Anderson and Roger Needham, “Programming Satan’s Computer,” in van Leeuwen, J. (eds) Computer Science Today, Lecture Notes in Computer Science, vol 1000, 1995. Springer, Berlin, Heidelberg.
- Roger M. Needham and Michael D. Schroeder, “Using Encryption for Authentication in Large Networks of Computers,” Communications of the ACM, Volume 21, Issue 12, 1978.
- Michael Burrows, Martín Abadi and Roger Needham, “A Logic of Authentication”, in Proceedings of the Royal Society of London A v 426, pp 233-271, 1989.
- Gavin Lowe, “An attack on the Needham-Schroeder public-key authentication protocol,” Information Processing Letters, Volume 56, Issue 3, 10 November 1995, Pages 131-133.
- Gavin Lowe, “Breaking and Fixing the Needham-Schroeder Public-Key Protocol using FDR,” in Margaria, T., Steffen, B. (eds) Tools and Algorithms for the Construction and Analysis of Systems. TACAS 1996. Lecture Notes in Computer Science, vol 1055. Springer, Berlin, Heidelberg.
- Ran Canetti and Jonathan Herzog, “Universally Composable Symbolic Security Analysis,” *Journal of Cryptology*, 24(1):83–147, 2011.